

Part 5: Visualization

Summary

This Part covers tools for visualizing data in R.

Powerful visualization techniques are critical to discovering and understanding the relationships between variables.

Example Data Set

First, we will revisit the data set we considered in the previous Part, and the steps needed to properly format the data:

```
> fulldata = read.table("q1_2017_all.csv", sep=",", header=T, quote="")
> fulldata$Date = as.Date(as.character(fulldata$Date), format="%Y%m%d")
> fulldata = fulldata[!duplicated(fulldata[,c(1,3)], fromLast=TRUE),]
> fulldata$McapRank = factor(fulldata$McapRank, ordered=TRUE)
> fulldata$TurnRank = factor(fulldata$TurnRank, ordered=TRUE)
> fulldata$VolatilityRank = factor(fulldata$VolatilityRank, ordered=TRUE)
> fulldata$PriceRank = factor(fulldata$PriceRank, ordered=TRUE)
```

The entire data set is a bit large to work with conveniently, so we are going to restrict consideration to 1000 equities:

```
> tickkeep = unique(fulldata$Ticker)[seq(1,
+      length(unique(fulldata$Ticker)), length=100)]
> fulldata = fulldata[fulldata$Ticker %in% tickkeep,]
```

1 Classic Plotting Functions in R

2 Prior to moving on the more advanced visualization functions, it is impor-
3 tant to understand the basic plotting capabilities of R.

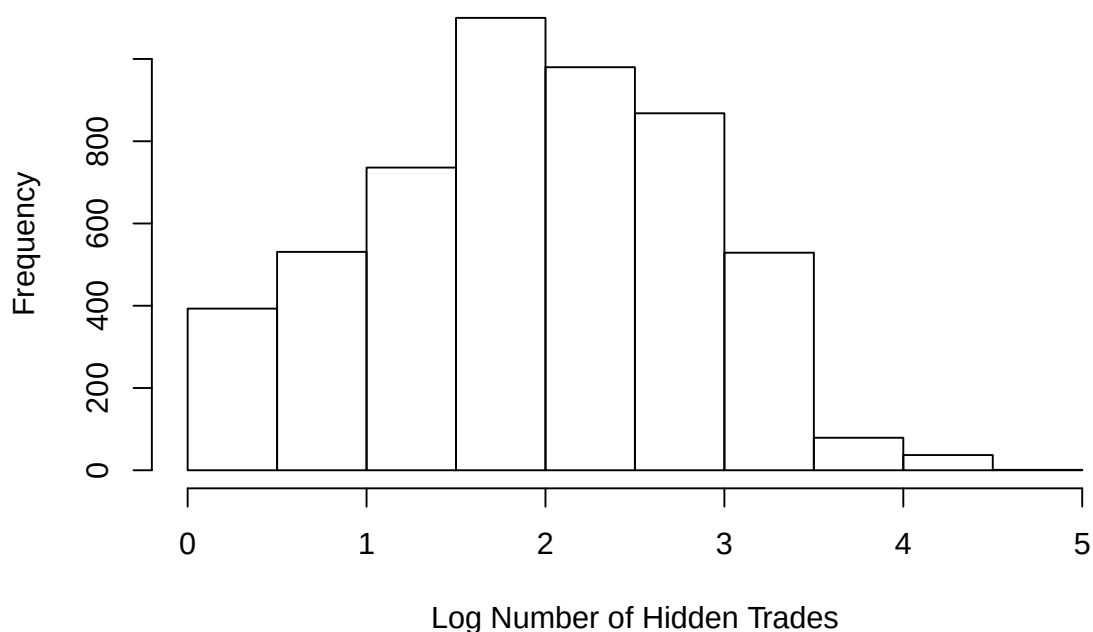
4 We saw the use of `boxplot()` in a previous Part. A similar view of the
5 shape of a distribution can be obtained using a **histogram**, formed using
6 the function `hist()`:

```
> hist(log10(fulldata$Hidden),  
+       xlab="Log Number of Hidden Trades")
```

7 The title on the plot can be changed using the `main` argument.

8 The number of bins in the histogram can be altered using the `breaks` ar-
9 gument.

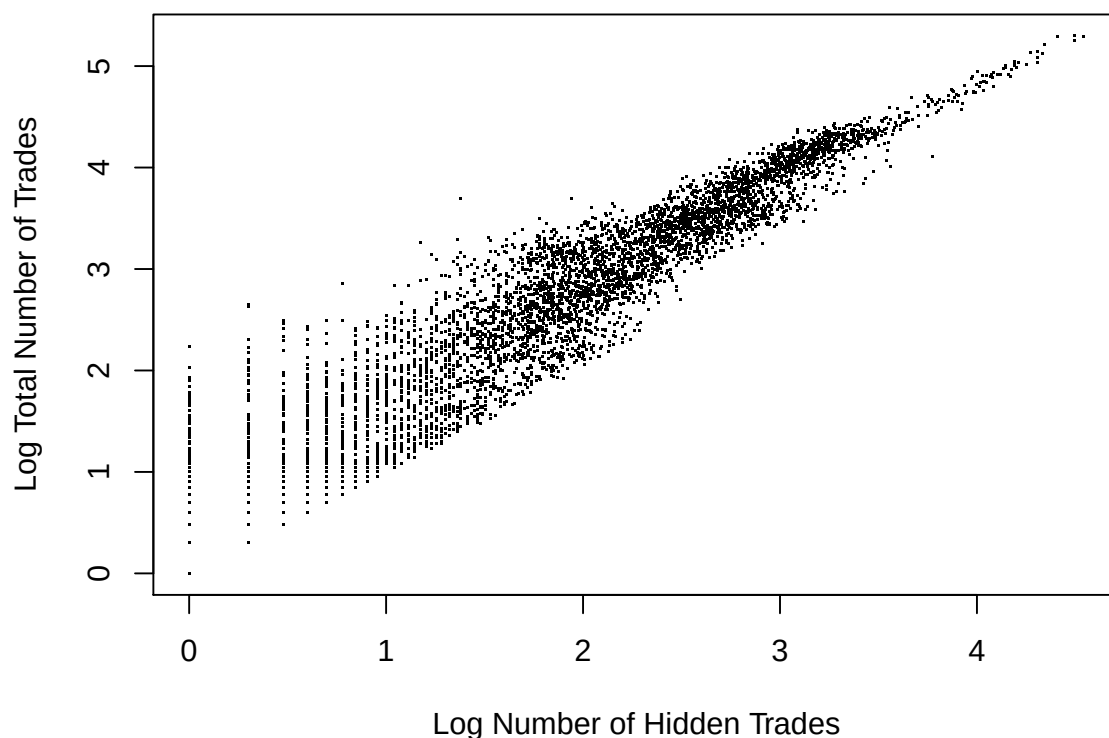
Histogram of `log10(fulldata$Hidden)`



- 1 **Scatter plots** are ubiquitous, as well. They are created simply in R using
- 2 the `plot()` function:

```
> plot(log10(fulldata$Hidden),  
+      log10(fulldata$TradesForHidden),  
+      xlab="Log Number of Hidden Trades",  
+      ylab="Log Total Number of Trades", pch=".")
```

- 3 The argument `pch` changes the plotting character. In a case such as this,
- 4 with so many dots on the plot, the default plotting character is too large.
- 5 Many other properties of the scatter plot can be adjusted using the `par()`
- 6 function. See `help(par)` for more information.



1 Introduction to ggplot2

2 The package `ggplot2` contains a great deal of valuable data visualiza-
3 tion tools. This package can be installed, along with other useful packages
4 for working with large and complex data sets, via the `tidyverse` meta-
5 package:

```
> install.packages("tidyverse")
```

6 The collection of packages in `tidyverse` comprise “a collection of R pack-
7 ages that share common philosophies and are designed to work together.”
8 – [tidyverse.org](https://www.tidyverse.org)

1 The `ggplot2` package implements a **grammar of graphics**, where plots are
2 created by combining **geoms**.

3 The syntax is a little awkward at first, but this approach yields a great deal
4 of flexibility, and a great deal of power in exploring features in a data set.

5 A plot is initialized by using the `ggplot()` function:

```
> ggplot(data = fulldata)
```

- 1 To the basic plot, one adds the “aesthetic” via the `mapping` option. An
2 aesthetic is a generic concept for the visual features of a plot. In this case, we
3 are specifying that the variable on the horizontal axis is `Hidden`, and the
4 variable on the vertical axis is `Cancels`:

```
> ggplot(data = fulldata, mapping =  
+         aes(x=Hidden, y=Cancels))
```

- 5 We can now add onto this base to create plots with complex features. You
6 can, for instance, define

```
> baseplot = ggplot(data = fulldata, mapping =  
+             aes(x=Hidden, y=Cancels))
```

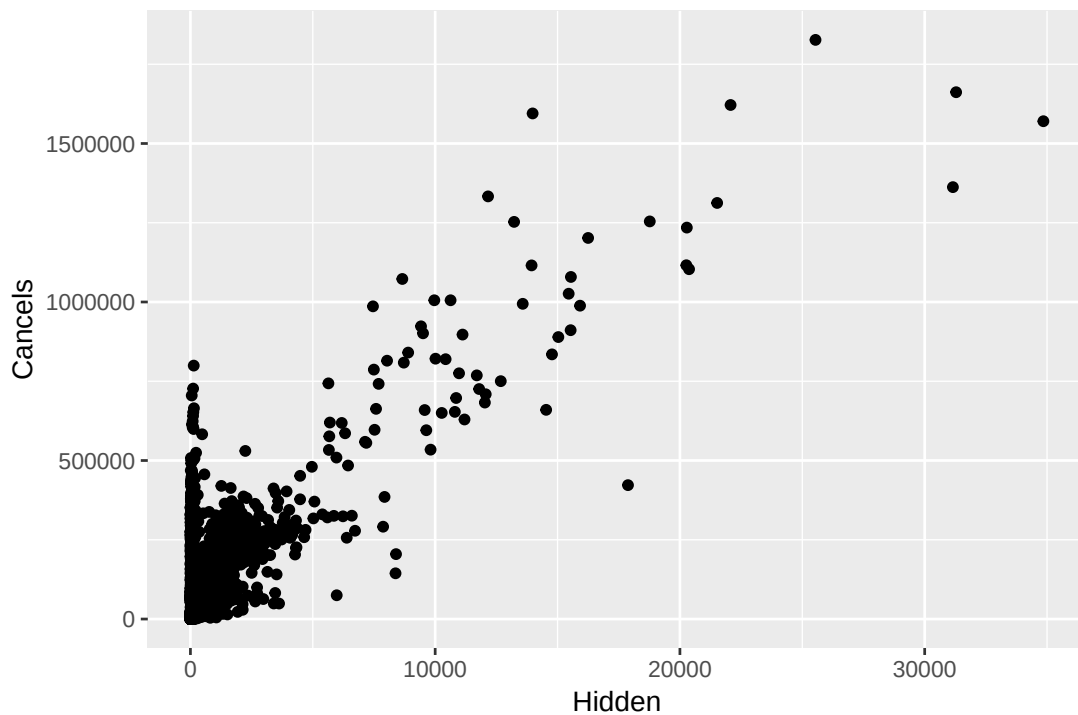
- 7 and add onto `baseplot` in what follows.

1 A Simple Scatter Plot

- 2 Consider the following syntax:

```
> baseplot + geom_point()
```

- 3 Note how the `geom_point()` function is used to add a scatter plot onto
4 the current figure, as stored in `baseplot`.



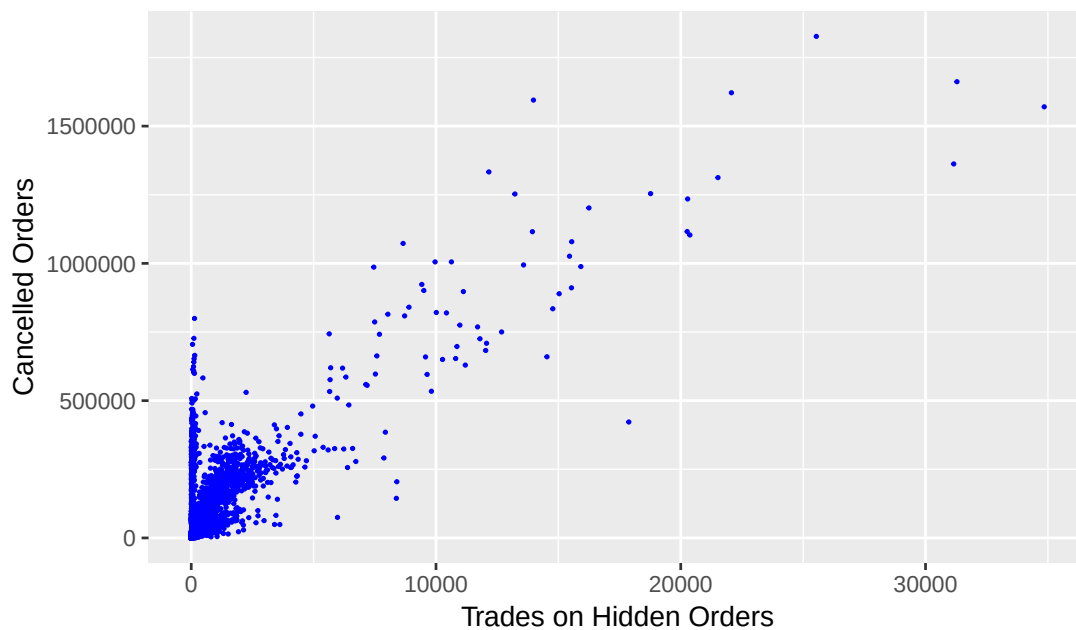
1

Now we will work to improve the appearance of this plot.

```
> baseplot +  
+   geom_point(size=0.3, color="blue") +  
+   labs(x="Trades on Hidden Orders",  
+        y="Cancelled Orders",  
+        title="Cancelled Orders versus Hidden Orders",  
+        subtitle="Daily, First Quarter of 2017")
```

Cancelled Orders versus Hidden Orders

Daily, First Quarter of 2017



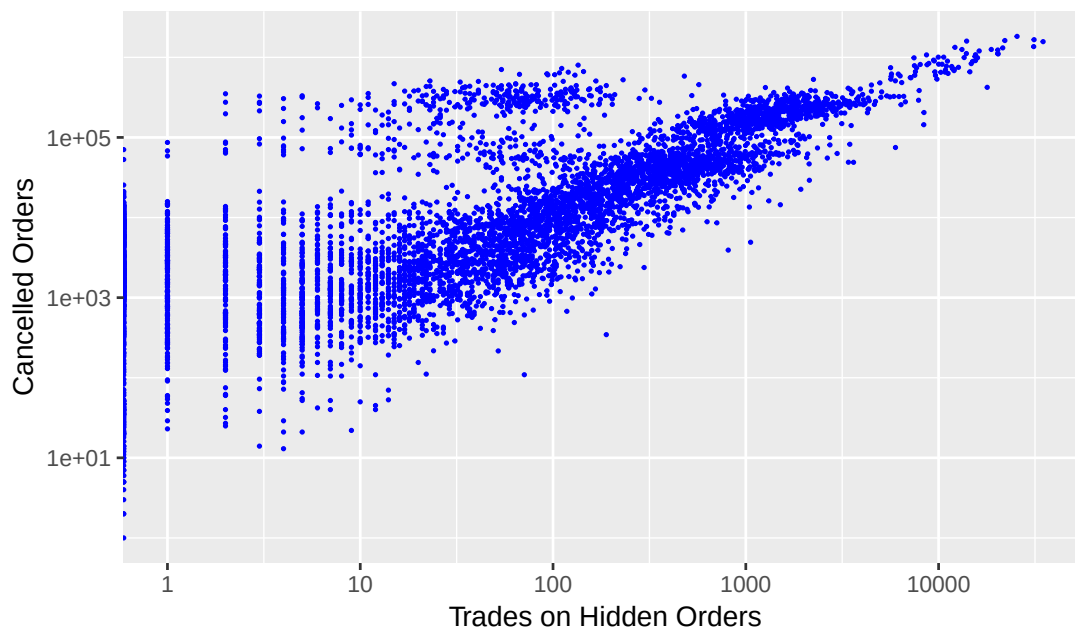
1

Next, we will convert the axes to the log scale:

```
> baseplot +  
+   geom_point(size=0.3, color="blue") +  
+   labs(x="Trades on Hidden Orders",  
+        y="Cancelled Orders",  
+        title="Cancelled Orders versus Hidden Orders",  
+        subtitle="Daily, First Quarter of 2017") +  
+   scale_x_log10() + scale_y_log10()
```

Cancelled Orders versus Hidden Orders

Daily, First Quarter of 2017



1

- 1 The `color` argument can be switched to vary with the levels of one of the
- 2 factors.

```
> baseplot +  
+ geom_point(size=0.3, mapping=aes(color=Security)) +  
+ labs(x="Trades on Hidden Orders",  
+      y="Cancelled Orders",  
+      title="Cancelled Orders versus Hidden Orders",  
+      subtitle="Daily, First Quarter of 2017") +  
+ scale_x_log10() + scale_y_log10()
```


Cancelled Orders versus Hidden Orders

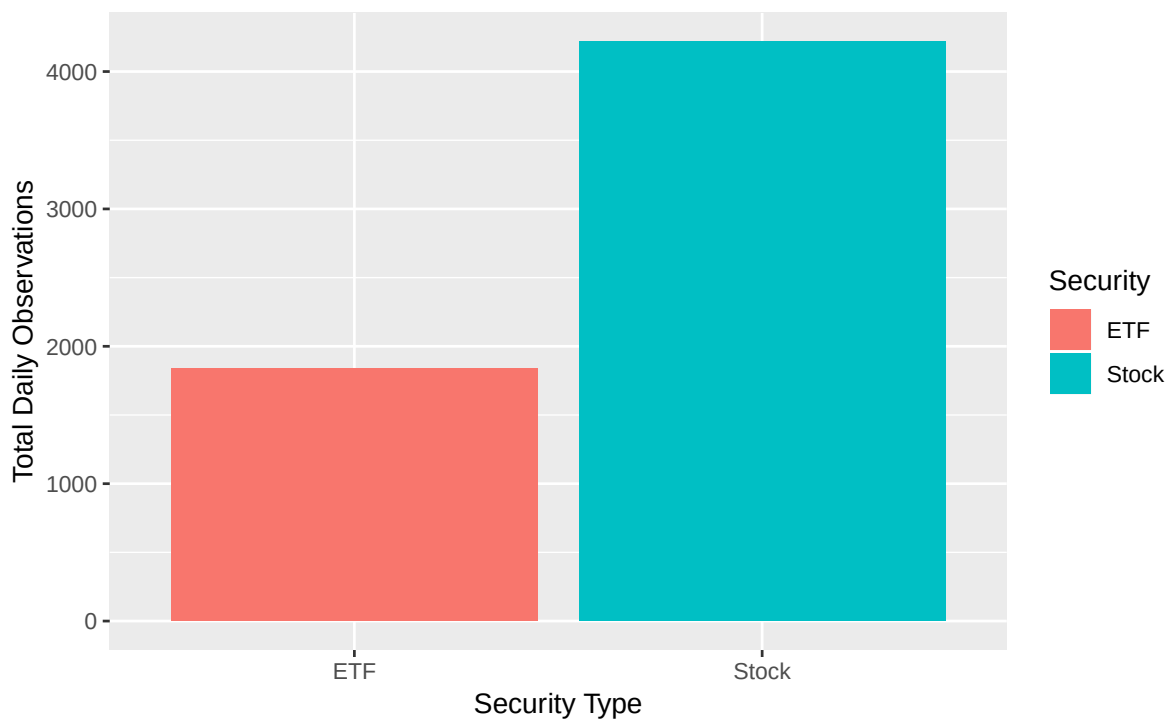
Daily, First Quarter of 2017



Bar Plots

- 2 Basic bar charts can be created to inspect the distribution of a factor, using
- 3 the `geom_bar()` function:

```
> ggplot(data=fulldata,  
+   mapping=aes(x=Security, fill=Security)) +  
+   geom_bar() +  
+   labs(x="Security Type",  
+   y="Total Daily Observations")
```



1

- 1 For the next plot, we will restrict to data that do not have missing values
- 2 on `McapRank`, and are of type "Stock":

```
> fulldataStock = filter(fulldata,  
+   Security == "Stock" & !is.na(McapRank))
```

- 3 We will adjust the aesthetic so that the "fill" color of the bars corresponds
- 4 to the level of `VolatilityRank`:

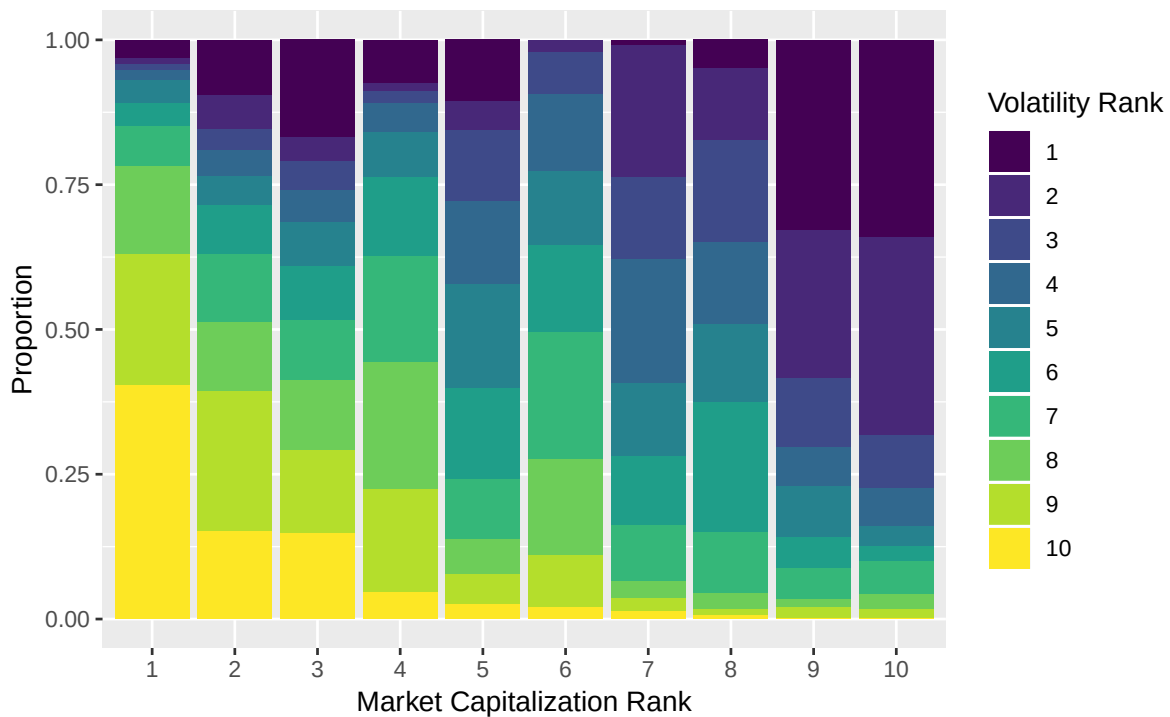
```
> ggplot(data=fulldataStock,  
+   mapping=aes(x=McapRank, fill=VolatilityRank)) +  
+   geom_bar() +  
+   labs(x="Market Capitalization Rank", y="Count",  
+   fill="Volatility Rank")
```



1

- 1 The bars in the previous plot are not of equal height because of the sub-
- 2 sampling we did to reduce to 1000 equities.
- 3 We can change vertical axis to proportion by setting `position="fill"`:

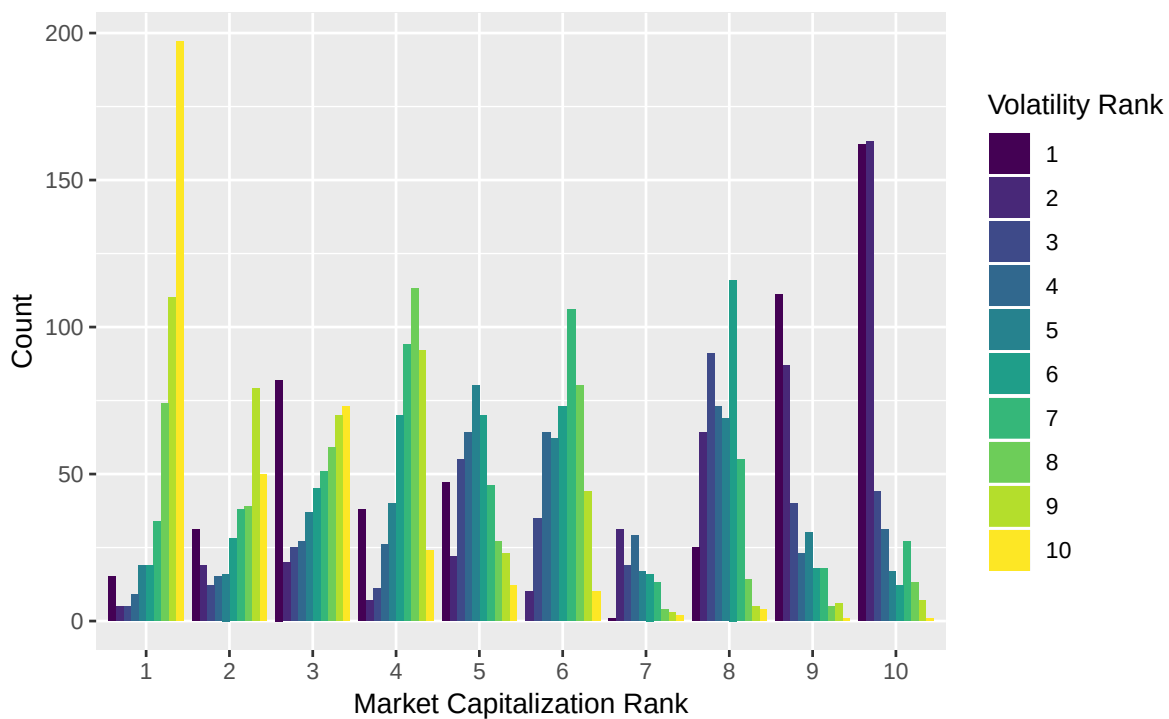
```
> ggplot(data=fulldataStock,  
+   mapping=aes(x=McapRank, fill=VolatilityRank)) +  
+   geom_bar(position="fill") +  
+   labs(x="Market Capitalization Rank",  
+   y="Proportion", fill="Volatility Rank")
```



1

- 1 By setting `position="dodge"`, the bars are arranged more like a his-
- 2 togram:

```
> ggplot(data=fulldataStock,  
+ mapping=aes(x=McapRank, fill=VolatilityRank)) +  
+ geom_bar(position="dodge") +  
+ labs(x="Market Capitalization Rank",  
+ y="Count", fill="Volatility Rank")
```

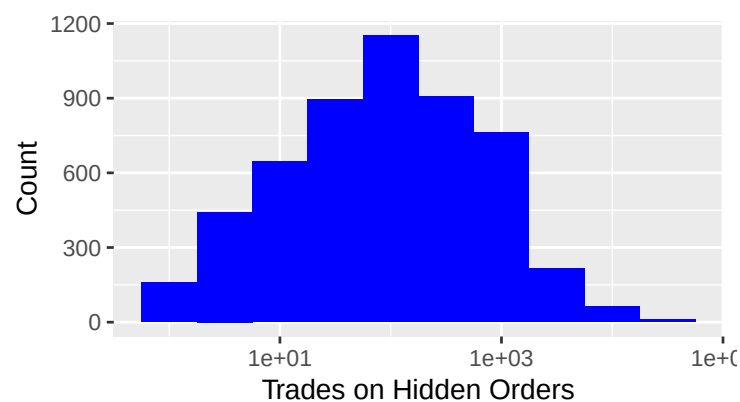


1

1 Histograms

2 The function `geom_histogram()` will create a histogram:

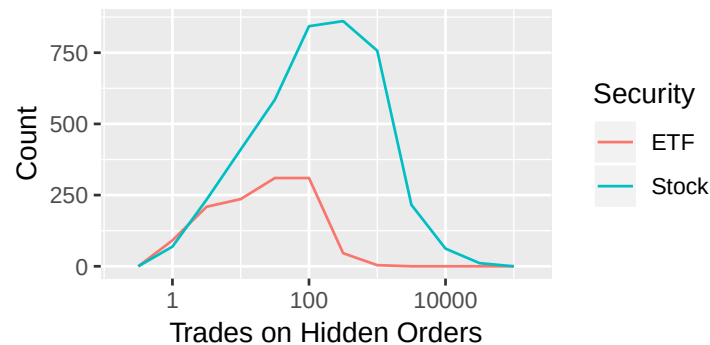
```
> ggplot(data=fulldata, mapping=aes(x=Hidden)) +  
+   geom_histogram(binwidth=0.5, fill="blue") +  
+   scale_x_log10() +  
+   labs(x="Trades on Hidden Orders", y="Count")
```



3

- 1 Overlapping histograms can be difficult to interpret (try it!). An alternative
- 2 is to create **frequency polygons** using `geom_freqpoly()`:

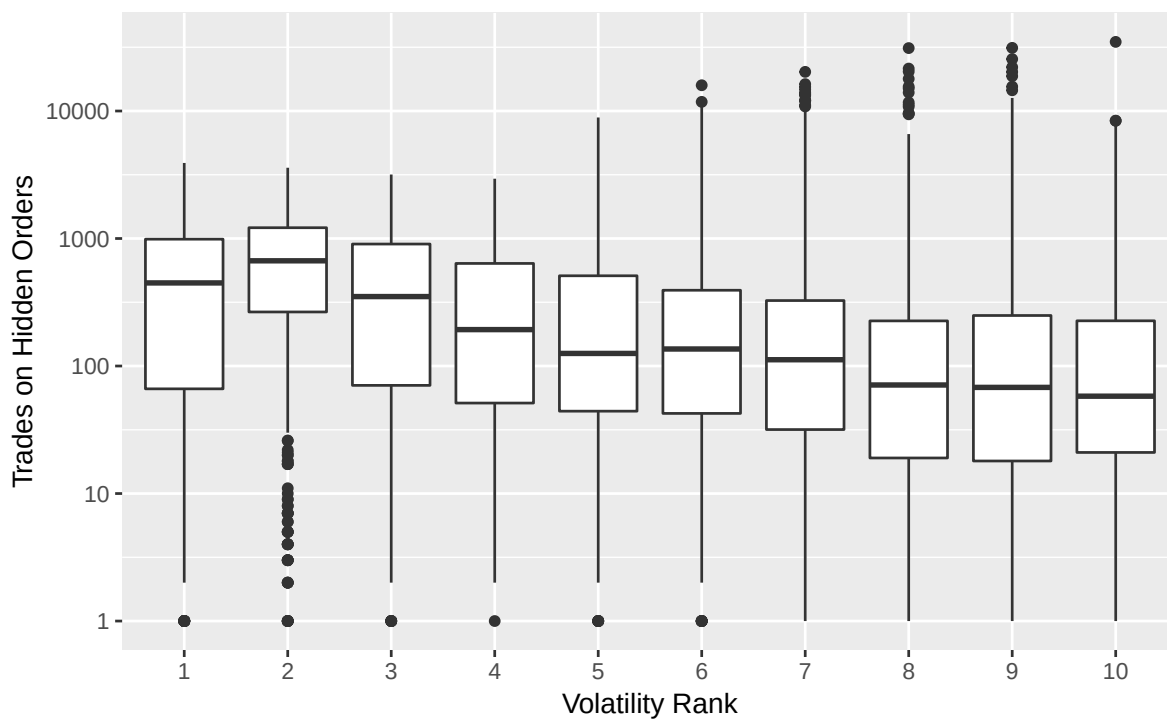
```
> ggplot(data=fulldata,
+         mapping=aes(x=Hidden, color=Security)) +
+   geom_freqpoly(binwidth=0.5, fill="blue") +
+   scale_x_log10() +
+   labs(x="Trades on Hidden Orders", y="Count")
```



3

- 1 **Boxplots**
- 2 **Side-by-side boxplots** are also useful for comparing the distributions of
- 3 variables over different values of a factor. For example,

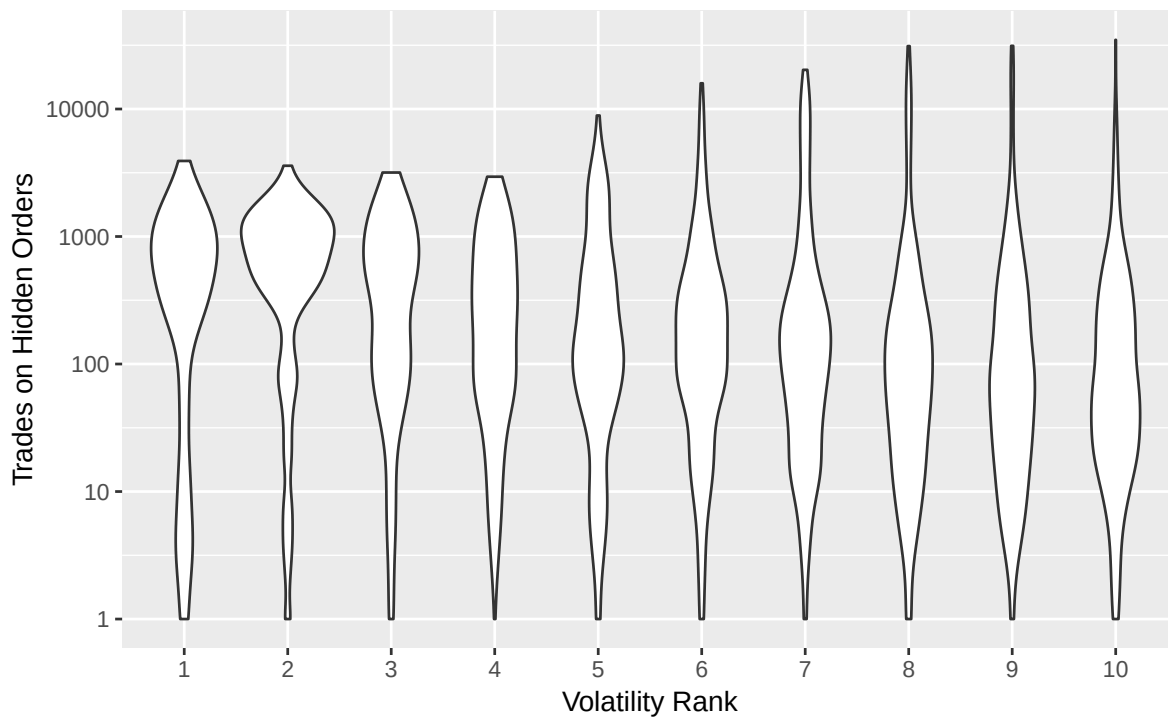
```
> ggplot(data=fulldataStock,
+         mapping=aes(x=VolatilityRank, y=Hidden)) +
+   geom_boxplot() + scale_y_log10() +
+   labs(y="Trades on Hidden Orders",
+        x="Volatility Rank")
```



1

- 1 A **violin plot** shows the same type of information, with a little more detail.
- 2 The width of a “violins” is greater in areas where a greater proportion of
- 3 the observations lie.

```
> ggplot(data=fulldataStock,  
+       mapping=aes(x=VolatilityRank, y=Hidden)) +  
+   geom_violin() + scale_y_log10() +  
+   labs(y="Trades on Hidden Orders",  
+       x="Volatility Rank")
```

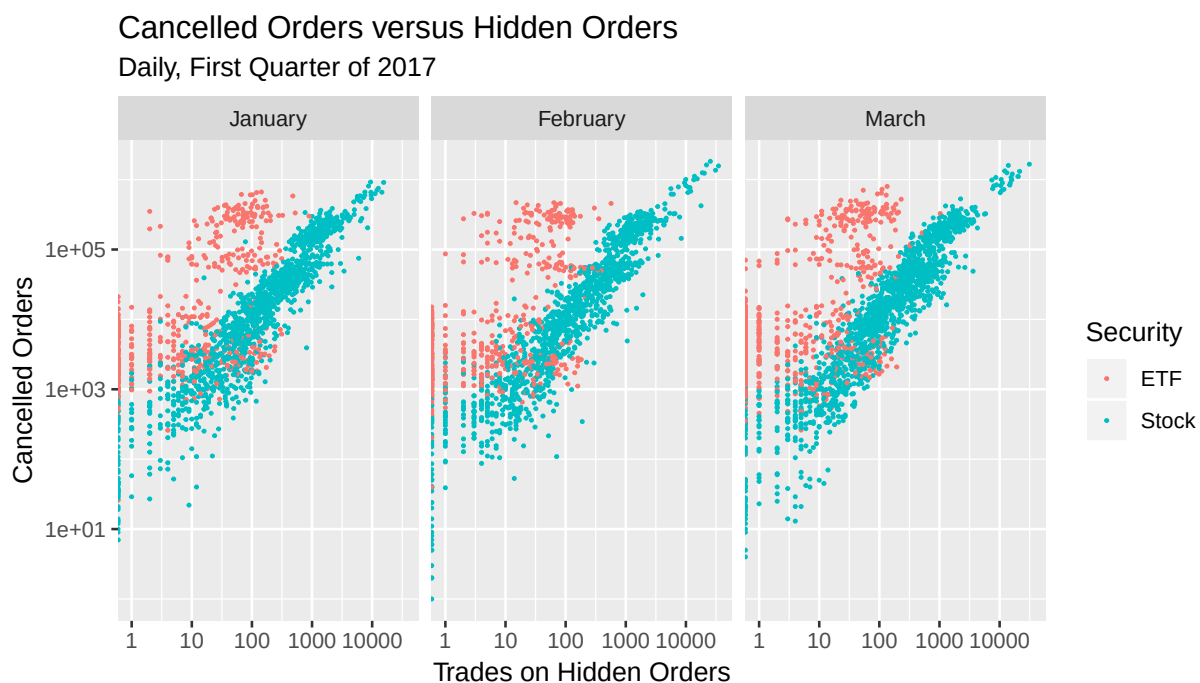


1

1 Facets

- 2 **Facets** refer to an arrangement of plots on which one or more factors
 3 vary. For example, return to our scatter plot example above, and use the
 4 `facet_grid()` function to break up the plots by month:

```
> baseplot +
+   geom_point(size=0.3, mapping=aes(color=Security)) +
+   labs(x="Trades on Hidden Orders",
+        y="Cancelled Orders",
+        title="Cancelled Orders versus Hidden Orders",
+        subtitle="Daily, First Quarter of 2017") +
+   scale_x_log10() + scale_y_log10() +
+   facet_grid(.~factor(months(Date),
+                        levels=c("January", "February", "March")))
```

1 **Exercise:** What happens in the previous example if the command

```
> facet_grid(.~months(Date))
```

2 is used instead? What about if use the following?

```
> facet_grid(months(Date)~.)
```

3

4

5

6

7

8

9

- 1 Two factors can vary:

```
> baseplot +
+   geom_point(size=0.3, color="red") +
+   labs(x="Trades on Hidden Orders",
+        y="Cancelled Orders",
+        title="Cancelled Orders versus Hidden Orders",
+        subtitle="Daily, First Quarter of 2017") +
+   scale_x_log10() + scale_y_log10() +
+   facet_grid(Security~factor(months(Date),
+                               levels=c("January", "February", "March")))
```



1 Time Series Plots

- 2 We will, of course, be working with a great deal of **time series data**. The
- 3 `geom_line()` function is useful:

```
> ggplot(data=filter(fulldata, Ticker=="AMD"),  
+         mapping=aes(x=Date, y=Hidden)) +  
+   geom_line(color="blue") +  
+   labs(x="Date", y="Trades on Hidden Orders",  
+         title="Change in Hidden Orders over Time",  
+         subtitle="AMD, daily, first quarter of 2017")
```

